



Actividad SENA: Consumo de API mediante Fetch en React con Vite

Descripción general

Esta actividad tiene como propósito que los aprendices desarrollen una aplicación básica en React con Vite para consumir datos desde una API pública utilizando `fetch()`, analizar el funcionamiento del código y comprender la diferencia entre API y API REST

La API Fetch proporciona una interfaz para solicitar recursos por red y devuelve una promesa con la respuesta, mientras que REST corresponde a un conjunto de restricciones de arquitectura para sistemas distribuidos, por lo que no toda API es necesariamente una API REST

Objetivo de aprendizaje

Al finalizar la actividad, el aprendiz estará en capacidad de construir un componente en React que consulte una API pública, procese la respuesta en formato JSON, represente los datos en la interfaz y explique conceptualmente las diferencias entre una API general y una API REST

Instrucciones para el aprendiz

1. Crear un proyecto en React usando Vite desde la terminal.
2. Ejecutar el proyecto y verificar que funciona correctamente en el navegador .
3. Consumir una API pública usando `fetch()`.
4. Mostrar en pantalla los datos obtenidos.
5. Implementar estados de carga, éxito y error en la aplicación
6. Analizar el código y explicar el papel de `useState`, `useEffect` y `fetch()` dentro del componente
7. Investigar qué es una API y qué es una API REST.
8. Elaborar una comparación donde se logre determinar la diferencia entre API y API REST
9. Proponer cómo esta implementación puede adaptarse posteriormente al proyecto grupal.

Competencia: Desarrollar la solución de software

Rap: Construir componentes Frontend

Desarrollo de la actividad

Paso 1. Crear el proyecto con Vite

Ejecutar en consola:

```
npm create vite@latest mi-app-fetch -- --template react
cd mi-app-fetch
npm install
npm run dev
```

Este flujo corresponde al inicio habitual de un proyecto React con Vite y permite levantar un entorno de desarrollo rápido para trabajar el consumo de datos

Paso 2. Reemplazar el contenido de App.jsx

Copiar y pegar el siguiente ejemplo comentado:

```
import { useEffect, useState } from "react";

function App() {
  // Estado donde se almacenan los usuarios que llegan desde la API
  const [usuarios, setUsuarios] = useState([]);

  // Estado para indicar si la información está cargando
  const [cargando, setCargando] = useState(true);

  // Estado para guardar posibles errores
  const [error, setError] = useState("");

  useEffect(() => {
    // Función asíncrona para consultar la API
    const obtenerUsuarios = async () => {
      try {
        const respuesta = await fetch("https://jsonplaceholder.typicode.com/users");

        // Se valida si la respuesta HTTP fue correcta
        if (!respuesta.ok) {
```

Competencia: Desarrollar la solución de software**Rap: Construir componentes Frontend**

```
    throw new Error("No se pudo obtener la información");
  }

  // Se transforma la respuesta a formato JSON
  const datos = await respuesta.json();

  // Se guardan los datos en el estado
  setUsuarios(datos);
} catch (err) {
  // Si ocurre un error, se almacena el mensaje
  setError(err.message);
} finally {
  // Finaliza el estado de carga
  setCargando(false);
}
};

// Llamado a la función al cargar el componente
obtenerUsuarios();
}, []);

return (
  <div style={{ padding: "20px", fontFamily: "Arial" }}>
    <h1>Consumo de API con Fetch en React</h1>

    {cargando && <p>Cargando usuarios...</p>}

    {error && <p style={{ color: "red" }}>Error: {error}</p>}

    {!cargando && !error && (
      <ul>
        {usuarios.map((usuario) => (
          <li key={usuario.id}>
            <strong>{usuario.name}</strong> - {usuario.email}
          </li>
        ))}
      </ul>
    )}
  </div>
);
}
```



Competencia: Desarrollar la solución de software

Rap: Construir componentes Frontend

```
export default App;
```

Paso 3. Análisis del ejemplo

- `useState` permite almacenar datos y actualizar la interfaz cuando cambian los valores del componente, por lo que resulta útil para guardar usuarios, mensajes de carga y errores
- `useEffect` permite ejecutar lógica cuando el componente se monta, lo que facilita hacer la consulta una sola vez al iniciar la vista
- `fetch()` realiza la solicitud HTTP al servidor y retorna una promesa con un objeto `Response`
- `response.json()` convierte la respuesta en un objeto JavaScript utilizable dentro de la aplicación
- La propiedad `response.ok` debe validarse porque `fetch()` puede resolver la promesa incluso cuando existe un error HTTP

Actividad de consulta e investigación

El aprendiz debe realizar una breve consulta conceptual y responder las siguientes preguntas:

- ¿Qué es una API?
- ¿Qué es una API REST?
- ¿Cuáles son las principales diferencias entre una API y una API REST?
- ¿Por qué las aplicaciones web modernas consumen servicios mediante HTTP y `fetch()`?

Orientación para establecer diferencias

Concepto	Explicación
API	Es una interfaz que permite la comunicación entre sistemas, componentes o aplicaciones de software
API REST	Es una API diseñada bajo restricciones REST, normalmente usando recursos accesibles por HTTP
Relación	REST es un tipo particular de API, pero el término API es más amplio y general
Uso en React	React puede consumir APIs desde el navegador mediante <code>fetch()</code> , procesando respuestas y actualizando la interfaz según el estado de la solicitud



Competencia: Desarrollar la solución de software

Rap: Construir componentes Frontend

Evidencias a entregar

- Proyecto funcional en React con Vite
- Código fuente con consumo de API usando `fetch()`
- Captura de pantalla de la aplicación en ejecución.
- Documento breve con la investigación sobre API y API REST
- Reflexión donde se explique cómo podría integrarse este ejercicio al proyecto grupal.

Criterios de evaluación

Criterio	Descripción
Configuración del entorno	Crea y ejecuta correctamente un proyecto en React con Vite
Consumo de API	Realiza una solicitud correcta mediante <code>fetch()</code> y procesa la respuesta
Manejo de estados	Implementa carga, error y visualización de datos
Comprensión conceptual	Diferencia adecuadamente entre API y API REST
Aplicación al proyecto grupal	Relaciona la práctica desarrollada con una necesidad real del proyecto formativo.

Recomendaciones

Se recomienda primero ejecutar el ejemplo propuesto, luego lo analicen línea por línea y finalmente lo adapten, por ejemplo, cambiando la URL de consulta, mostrando otros campos o creando una interfaz más cercana a la necesidad de su proyecto grupal